Outlook

Comments on the RDF/LPG analysis and RWDKN Neo4j implementation findings

From Kayaalp, Mehmet (NIH/NLM/LHC) [E] <mkayaalp@mail.nih.gov>
Date Thu 11/5/2026 4:55 PM
To Kola, Chandra (NIH/NLM) [C] <kolac@mail.nlm.nih.gov>

Hi Chandra,

Thank you for putting together the comparison table and the accompanying webpage, *Knowledge Graph Paradigms for Clinical Decision Support: LPG (Neo4j) vs. RDF (OWL/SHACL)*. I think undertaking this kind of analysis is a very good first step. In particular, grounding the architectural discussion in actual RWDKN artifacts—query templates, loaders, missed results, validation failures, and maintenance code—gives us something concrete to examine rather than relying only on abstract claims about RDF and LPG. The webpage explicitly positions itself as an "architectural and empirical evaluation" based on the RWDKN pipeline, which is exactly the direction I think our evaluation should take.

At the same time, I think several of the shortcomings identified in the webpage need to be reformulated. The examples do identify real shortcomings, but in many cases they are shortcomings of **our current RWDKN Neo4j implementation**, rather than intrinsic shortcomings of Neo4j or the LPG model. We should preserve the genuine RDF/OWL advantages—for example, RDF Schema (RDFS) formally defines rdfs:subClassOf as transitive and states that instances of a subclass are also instances of its superclass. But we should distinguish those semantic advantages from cases where Neo4j already supports the required database/query capability and our implementation either did not use it or implemented it in a brittle way.

I would characterize and address the RWDKN implementation shortcomings as follows.

1. Inconsistent class assignment during KGX-to-Neo4j projection

In Section 2 of the webpage, the Semaglutide example is traced to the fact that export_kgx.py uses a "first-seen-wins node deduplication" strategy, resulting in different GLP-1 entities receiving inconsistent categories across sources. Semaglutide and Liraglutide are shown as ChemicalEntity only, while Tirzepatide and Dulaglutide may receive Drug in one source and ChemicalEntity in another.

This appears to be an integration/projection problem rather than a Neo4j limitation. There is also an important semantic point here. RDFS states that if class C is a subclass of C′, "all instances of C will also be instances of C′." Therefore:

Drug → ChemicalEntity

does not imply:

ChemicalEntity → Drug.

If Semaglutide is assigned only the broader ChemicalEntity category upstream, OWL/RDFS subclass reasoning alone cannot infer that it is a Drug.

Suggested remedy: Replace first-seen-wins typing with a deterministic canonical typing and reconciliation policy. Preserve multiple valid Biolink categories where appropriate and/or materialize applicable ancestor categories during ingestion. Add source-order-independent regression tests so that the same CURIE receives the same canonical typing regardless of which source is processed first.

2. Artificial one-hop limitation in subclass traversal

Section 3 attributes the omission of Xultophy to this Cypher pattern:

[:SUBCLASS_OF*0..1]

The webpage itself explains that the query therefore fails to reach Xultophy at two hops.

That is a query-template limitation, not a Neo4j traversal limitation. Current Neo4j documentation states that “Cypher can be used to match patterns of a variable or an unknown length” and specifically identifies traversing “a hierarchy that may have differing depths” as a use case.

Suggested remedy: Use the current quantified-relationship syntax rather than imposing a one-hop bound. For the relationship type already used by the RWDKN implementation, the basic pattern would be:

```
MATCH (x)-[:SUBCLASS_OF]->+(ancestor)
RETURN x, ancestor
```

The + quantifier represents one or more relationships. Neo4j describes a quantified relationship as “an abbreviated form of a quantified path pattern” and documents -[]->+ as the modern equivalent of a one-or-more variable-length relationship. The older * variable-length syntax remains supported, but Neo4j notes that it “is not GQL conformant.”

If unrestricted traversal raises performance concerns, we should use relationship, node, and property predicates to prune the search rather than arbitrarily truncating the ontology hierarchy. Neo4j specifically documents inline predicates for this purpose.

3. Hard-coded enumeration of Biolink child classes

Also in Section 3, the RWDKN query template explicitly enumerates:

Disease|ClinicalFinding|Phenotype|PhenotypicFeature

rather than deriving the relevant classes from a maintained hierarchy.

The webpage is correct that this design is brittle: adding or changing a relevant Biolink subclass can require modifying the query templates. The intrinsic RDF advantage is that RDFS provides formal subclass semantics—the W3C specification states that rdfs:subClassOf “is transitive.” However, manually listing every subclass in every Cypher template is not an intrinsic requirement of Neo4j.

Suggested remedy: Represent the approved Biolink category hierarchy explicitly in the LPG and centralize hierarchy expansion. Alternatively, materialize applicable ancestor categories on entity nodes during ingestion. The query translator should obtain the subclass closure from this central semantic

representation rather than contain independently maintained label lists. Updating the Biolink profile would then update the hierarchy once rather than require edits to multiple Cypher templates.

4. Hard-coded predicate lists and the omitted RELATED_TO relationship

Section 4 identifies another concrete RWDKN query-template problem: _ADVERSE_EVENT_REL_PATTERN contains a hard-coded list of predicates and omits RELATED_TO, causing a PubMed Semaglutide–Nausea relationship to be missed.

The absence of native RDFS subPropertyOf entailment is a genuine semantic distinction. RDFS states that rdfs:subPropertyOf expresses that “all resources related by one property are also related by another,” and the relation itself is transitive.

But the omission of RELATED_TO from a manually maintained string is an implementation defect. In addition, the webpage's statement that Cypher “cannot parameterize” relationship types is not correct for current Neo4j. Its documentation states: “Node labels and relationship types can be referenced dynamically in expressions, parameters, and variables.”

Suggested remedy: Maintain one machine-readable predicate hierarchy derived from the approved Biolink predicate profile. Compute the relevant descendant/super-predicate closure centrally and pass the resulting relationship types dynamically to the query. Where frequently used semantic closure relationships are stable and performance-critical, we can also evaluate materializing them during ingestion. What we should avoid is maintaining independent pipe-delimited predicate lists across query templates.

5. Semantic expansion logic embedded independently in individual query templates

The child-class and predicate examples above point to a broader architectural issue in our Neo4j implementation: semantic expansion has been placed directly into individual Cypher templates. The webpage itself shows both manual class enumeration and manually maintained predicate lists.

I agree with the underlying concern that ontology semantics should not have to be independently reconstructed by every query. RDF/OWL has a genuine advantage in allowing certain class, property, and relationship semantics to be declared once and applied through reasoning. But the alternative in Neo4j does not need to be duplicated traversal logic throughout the application.

Suggested remedy: Move semantic expansion into a centralized semantic query-planning layer. That layer should resolve Biolink classes, descendants, predicates, inverses, and other supported semantic transformations before constructing the Cypher query. Reusable graph patterns or materialized semantic shortcut relationships can then handle recurring traversal requirements. This would not turn Neo4j into an OWL reasoner, but it would eliminate much of the brittleness currently caused by distributing semantic knowledge among individual query templates.

6. Use of OPTIONAL MATCH for a relationship that was functionally mandatory

Section 5 reports that all 19 RegimenChoiceGroups were loaded but became detached from their intended clinical associations because the loader used OPTIONAL MATCH.

Current Neo4j documentation explains the relevant behavior directly: “if no matches are found, OPTIONAL MATCH will use a null for missing parts of the pattern.” Its cheat sheet makes the intended distinction even clearer: “Use MATCH to find entities that must be present in the pattern. Use OPTIONAL MATCH to find entities that may not be present.”

If the Association node is required for a valid RegimenChoiceGroup, using optional semantics without an enforced postcondition allowed the loader to continue when it should have failed.

Suggested remedy: Use mandatory MATCH for graph structures that are required by the data model, or explicitly abort the transaction when the required association cannot be resolved. Add transaction-level/post-load assertions for required connectivity and retain the existing integrity tests as defense in depth.

7. Insufficient use of Neo4j's native schema and integrity mechanisms

Section 5 characterizes the custom Python validation code as necessary “because Neo4j has NO native schema/shape validation.” That statement is too broad for current Neo4j.

Neo4j documentation states that it “offers several constraints to ensure the quality and integrity of data in a graph,” including uniqueness, property-existence, property-type, and key constraints. Current Neo4j also allows an open schema to be imposed; its documentation says Graph Types provide “a defined structure for an entire graph.”

SHACL remains more expressive and portable for many graph-shaped constraints, and that should be credited as an RDF-stack advantage. But that is different from concluding that Neo4j has no native integrity mechanisms.

Suggested remedy: Define the RWDKN invariants explicitly and divide them into: (a) requirements enforceable by Neo4j constraints/Graph Types in the deployed version, and (b) higher-order graph constraints requiring a validation layer. Enforce category (a) directly at the database boundary and use external or application-level validation only for category (b). If the deployed Neo4j version predates Graph Types, that should be recorded as a version-specific implementation constraint.

8. Query/application complexity is being attributed to the graph paradigm without equivalent baselines

Section 6 reports approximately 766 lines for the RDF/OWL/SHACL stack versus 6,964 lines for the Neo4j side. However, the webpage shows that the Neo4j total includes a 4,207-line query_app.py, loader, adapter, semantic query planner, Cypher templates, and query-generation/retry code, while the RDF total consists primarily of RDF projection, SHACL validation, and shape definitions.

Those are not equivalent software boundaries, so this comparison cannot by itself tell us how much additional code is intrinsically required by LPG versus RDF.

Suggested remedy: Repeat the code-footprint comparison using like-for-like components: ingestion versus ingestion, semantic mapping versus semantic mapping, validation versus validation, query-planning versus query-planning, and application layer versus application layer. We should separately

identify code necessitated by the graph representation from code implementing application requirements that either architecture would need.

Overall, I think the webpage has identified several useful and actionable weaknesses in the current RWDKN Neo4j implementation. I would not discard those findings; I would reclassify them. In particular, we should distinguish:

1. **Intrinsic semantic differences** — for example, standardized RDFS/OWL class and predicate entailment that Neo4j does not itself provide;
2. **Neo4j capabilities that our current implementation has not used effectively** — such as arbitrary-depth quantified traversal, dynamic relationship types, and database integrity mechanisms; and
3. **RWDKN-specific implementation defects** — such as first-seen-wins classification, one-hop query bounds, hard-coded class/predicate lists, and optional loading of mandatory structures.

I think the most informative next step would be to correct categories 2 and 3 and then repeat the same competency questions and audit tests. Whatever differences remain after that exercise would give the Tech WG much stronger evidence about the intrinsic architectural trade-offs between RDF/OWL and LPG.

Best,

Mehmet Kayaalp, M.D., Ph.D.
Staff Scientist/Medical Officer
Applied Clinical Informatics Branch
LHNCBC/NLM
National Institutes of Health
Bldg 38A, Rm 7S714
301-827-5017
mehmet.kayaalp@nih.gov